Efficient Sequence Modeling & Attention Mechanisms

Junpeng Zhao
UC San Diego
juz063@ucsd.edu

William Liu
UC San Diego
wil017@ucsd.edu

Kabir Vats
UC San Diego
kvats@ucsd.edu

Abstract

Scaling large language models to long contexts is increasingly constrained by inference-time memory rather than computation. This work investigates memory-efficient attention mechanisms through both a structured review and empirical evaluation. We compare multi-head (MHA), multi-query (MQA), and grouped-query attention (GQA) across several language models using two benchmarks: a performance test measuring Key–Value (KV) cache size and throughput, and a Needle-in-a-Haystack task under KV cache quantization. Results show that MQA achieves the lowest memory usage and most stable throughput, MHA incurs the highest cost, while GQA offers a practical trade-off. Smaller models are also more sensitive to low-precision KV caches. These findings clarify design trade-offs for long-context deployment under memory constraints.

1 Introduction

Auto-regressive transformers maintain a Key–Value (KV) cache that stores key and value tensors for previously generated tokens. While this cache enables efficient incremental decoding, its memory cost grows linearly with sequence length, number of layers, and hidden dimension. As context windows scale to tens or hundreds of thousands of tokens, the KV cache becomes a major inference-time memory bottleneck.

A variety of methods have been proposed to address this constraint, and they can be categorized as cache compression, sparse access, and architectural simplification.

In order to empirically examine some of the methods, we conducted a battery of tests to evaluate the performance of different KV cache attention methods, and also their performance under quantization. We first measure the KV cache size of models and token generation speed across a range of context lengths. We then conduct a Needle-in-a-Haystack test, where models' KV caches are quantized to different degrees. From the results of this test, we are able to determine many interesting properties of different KV cache designs.

2 Relevant Works

2.1 Cache Compression

Cache compression methods aim to reduce the memory footprint of these stored representations while preserving the information required for attention. Existing approaches compress the KV cache along three primary axes: precision reduction, dimensional compression, and token reduction, corresponding to KV quantization, low-rank KV compression, and token/KV merging.

KV quantization reduces memory usage by storing keys and values in lower numerical precision. Instead of standard 16- or 32-bit floating-point formats, KV tensors can be stored using lower-bit representations such as 8-bit or 4-bit values. Because the KV cache is frequently read but rarely modified during inference, it is well suited to quantization. Prior work on low-precision transformer inference shows that models can operate effectively with reduced numerical precision. For example, LLM.int8() demonstrates that transformer computations can be performed with 8-bit representations while maintaining strong performance across tasks (Dettmers et al., 2022). Similarly, QLoRA shows that 4-bit quantization can preserve model quality when appropriate scaling and calibration are used (Dettmers et al., 2023). While these works primarily focus on model weights, they motivate similar techniques for KV cache compression. KIVI introduces a tuning-free asymmetric quantization method that stores KV caches using 2-bit representations while maintaining generation quality across long contexts (Liu et al., 2024).

Low-rank compression instead exploits redundancy in transformer representations. Rather than storing full KV tensors, these methods approximate them using low-rank matrix factorizations that store a smaller set of basis vectors and coefficients. Empirical studies show that pretrained language models often operate in a low intrinsic dimension, suggesting substantial redundancy in their internal representations (Aghajanyan et al., 2021). By leveraging this property, low-rank KV compression can reduce memory requirements while preserving most of the information needed for attention. The rank controls the trade-off between compression efficiency and reconstruction accuracy, though reconstruction may introduce additional computation during attention.

A complementary approach reduces memory by decreasing the number of stored tokens rather than compressing individual vectors. Many tokens in long sequences produce highly similar key representations or attention contributions, making them redundant for future attention computations. Token Merging (ToMe) dynamically merges similar tokens by identifying tokens with similar representations and aggregating them into a smaller set (Bolya et al., 2023). Although originally proposed for vision transformers, similar token clustering or merging ideas can be applied to language models to reduce redundant KV representations in long-context inference. However, aggressive merging can degrade performance if important contextual distinctions are lost.

2.2 Sparse Access

In long-context processing with LLMs, the memory footprint of the KV cache increases linearly with the sequence length, posing a primary bottleneck for on-device inference. To manage this dynamically growing memory state and prevent Out-Of-Memory (OOM) errors, two major approaches have been extensively studied: Sparse Access methods, which restrict attention matrix computation and memory retrieval, and Eviction Strategies, which dynamically remove redundant cache entries.

Early Sparse Access methods reduced computational complexity and memory usage by associating tokens according to predefined, fixed patterns rather than computing the full attention matrix. Representative works include Sparse Transformer (Child et al., 2019) and Longformer (Beltagy et al., 2020), which combine "Sliding Window" mechanisms (focusing only on immediate neighboring tokens) with dilated or global patterns to capture broader context. However, these static heuristics can't dynamically determine which information within the context is useful for the future tokens. Consequently, they significantly limit the "Effective Context Window" of the model and degrade performance in retrieval-intensive tasks, such as the "Needle in a Haystack" test.

To overcome the limitations of static approaches, strategies have been proposed to dynamically evaluate the importance of each token during the generation process and selectively retain or evict KV cache entries. H2O (Heavy-Hitter Oracle) (Zhang et al., 2023), exploits the phenomenon where LLM attention scores concentrate heavily on a small subset of specific tokens, termed "Heavy-Hitters." By dynamically evicting KV cache entries with low cumulative attention scores, H2O significantly reduces the memory footprint while maintaining generation quality. Similarly, Scissorhands (Liu et al., 2023) operates on the "Persistence of Importance" hypothesis, demonstrating that tokens identified as important in previous steps tend to remain influential in future steps, thereby enabling efficient KV cache compression at inference time.

A known failure mode with sliding-window KV cache eviction is that generation quality can decrease drastically after the first few prompt tokens are evicted. Xiao et al. analyzed this and identified the "Attention Sink" phenomenon (Xiao et al., 2024), where transformers assign disproportionately high attention scores to the initial tokens even if they lack significant semantic value. Their proposed framework, StreamingLLM, employs a hybrid eviction strategy that persistently retains the KV cache of these attention sinks alongside a recent local window. In their paper, they demonstrate that this approach enables stable streaming generation over virtually infinite sequence lengths within finite memory constraints, without forgetting the crucial anchors at the beginning of the prompt (Xiao et al., 2024).

PagedAttention (Kwon et al., 2023) improves KV cache efficiency through memory layout optimization rather than direct compression. It partitions the KV cache into fixed-size pages and manages them through a virtual memory–style mapping, reducing fragmentation and improving memory utilization during batched inference.

2.3 Architectural Simplification of Attention Memory

Architectural simplification of memory-efficient attention focuses on reducing the number of distinct key and value projections that must be stored during autoregressive decoding. In a standard Transformer, multi-head attention (MHA) assigns separate query, key, and value projections to each attention head, which improves representational diversity but also increases the size of the KV cache linearly with the number of KV heads and with sequence length (Vaswani et al., 2017). For long-context generation, this design makes inference increasingly memory-bandwidth-bound, since the model must repeatedly read a large cache of historical keys and values at every new decoding step, motivating methods such as Multi-query attention (MQA) (Shazeer, 2019).

MQA is the most aggressive simplification of MHA. Instead of maintaining separate key and value heads for every query head, MQA shares a single key head and a single value head across all query heads (Shazeer, 2019). This change preserves multiple query projections, so the model still attends from several perspectives, but it drastically reduces the KV cache footprint and the amount of memory traffic required during decoding. Shazeer motivates MQA primarily as a systems-aware architectural change: inference in decoder-only models is often limited less by arithmetic than by repeatedly loading large key and value tensors from memory (Shazeer, 2019). By collapsing all KV heads into one, MQA directly targets that bottleneck.

The main trade-off is that MQA reduces representational flexibility. In MHA, each head can learn its own key and value subspace, allowing different heads to specialize in different linguistic or positional relationships. MQA forces all query heads to read from the same shared key-value representation, which lowers memory cost but can weaken quality relative to full MHA (Shazeer, 2019). Thus, MQA is best understood as minimizing cache size at the cost of representative diversity and specialization.

Grouped-query attention (GQA) was introduced as a compromise between these two extremes (Ainslie et al., 2023). Instead of using either one KV head for every query head (MHA) or one KV head for all query heads (MQA), GQA partitions query heads into groups, with each group sharing one key head and one value head.

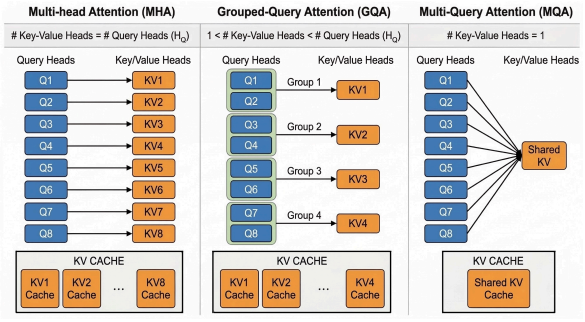


Figure 1: Structural comparison of MHA, GQA, and MQA.

This allows GQA to use more KV heads than MQA but less than MHA, and turns the number of KV heads into a design choice independent of the number of attention heads (Ainslie et al., 2023). As a result, GQA preserves some KV head-specific diversity while still shrinking the cache and reducing memory bandwidth demand. Empirically, Ainslie et al. argue that GQA attains quality close to MHA while retaining inference speed comparable to MQA, which makes it a practical architectural compromise for modern large language models (Ainslie et al., 2023).

Taken together, MHA, MQA, and GQA are three kinds of attention-mechanism architectural simplification that can improve KV cache efficiency. MHA offers maximum expressivity but has the largest KV cache cost. MQA uses the smallest cache and the strongest decoding-speed advantage, but may sacrifice accuracy because all heads share the same key-value memory. GQA is a tradeoff between the two prior methods. It reduces memory cost substantially while preserving more of the specialization that gives MHA its modeling power (Shazeer, 2019; Ainslie et al., 2023). The structural comparison of MHA, GQA, and MQA is shown in Figure 1.

3 Experimental Framework

We evaluated the impact of different attention mechanisms on inference efficiency and long-context robustness using a controlled benchmarking framework. We benchmarked models across MHA, MQA, and GQA families as seen in Table 1. These models allow us to compare how attention design influences memory usage and inference performance.

To analyze memory scaling, we measured KV cache size as a function of input sequence length prior to token generation. Because KV cache stores key and value tensors for each processed

token, memory usage is expected to grow linearly with sequence length.

We also evaluated inference throughput, measured in tokens per second, across different input sequence lengths. Throughput is normalized relative to performance at an input length of 200 tokens to highlight performance degradation as context increases.

Finally, we evaluate long-context robustness using a Needle-in-a-Haystack retrieval task, where a target phrase is embedded within long irrelevant text and the model must retrieve it. To study compression effects, we evaluate performance under KV cache quantization levels of 4-bit, 8-bit, and 16-bit precision.

All experiments were conducted on a Kaggle GPU environment with an NVIDIA Tesla P100.

3.1 Models Used

The models used in this study and their specifications are shown in Table 1.

Table 1: Model architecture comparison across attention mechanisms.

Model	Params	Attn	Heads	KV Heads
OPT	1.3B	MHA	32	32
Llama-2	7B	MHA	32	32
Phi-2	2.7B	MHA	32	32
Falcon	7B	MQA	71	1
Gemma	2B	MQA	8	1
Mistral	7B	GQA	32	8
Gemma-2	2B	GQA	8	4
Qwen-2.5	0.5B	GQA	14	2

3.2 Performance Benchmark

We evaluate model performance under two prompting regimes. In the first setting, input prompts range from 100 to 1900 tokens and the model generates 100 tokens. In the second setting, prompts contain 100 tokens while generation length varies from 100 to 1900 tokens.

For each configuration we measure the following metrics:

- **KV Cache Size (Prefill):** Memory usage of the KV cache after processing the input prompt, measured in MiB.
- **KV Cache Size (Final):** Memory usage of the KV cache after completing generation, measured in MiB.
- **Generation Throughput:** Average generation speed measured in tokens per second.

These measurements allow us to analyze how context length and generation length affect memory consumption and inference throughput.

3.3 Needle-in-a-Haystack Benchmark

In the Needle-in-a-Haystack benchmark, models are tasked with retrieving a passcode ("needle") hidden within a sequence of distractor tokens ("haystack"). We vary both the total context length (128–2048 tokens) and the needle position (10%–90% of the sequence). Each configuration is evaluated over three trials, for a total of 60 runs.

Models are tested under three KV cache quantization settings: no quantization (16-bit floating point), 8-bit quantization, and 4-bit quantization. Quantization is implemented using Half-Quadratic Quantization (HQQ) (Badri et al., 2023).

Evaluation checks whether the model's 20-token output contains the correct passcode. We use substring matching rather than exact matching, since small models often fail to strictly follow formatting instructions; the benchmark therefore prioritizes successful retrieval of the needle over exact output form.

4 Results

4.1 Performance Benchmark

Figure 2 illustrates KV cache size versus input length before generation for each model, whereas Figure 3 illustrates token generation speed versus input length normalized to 200 tokens.

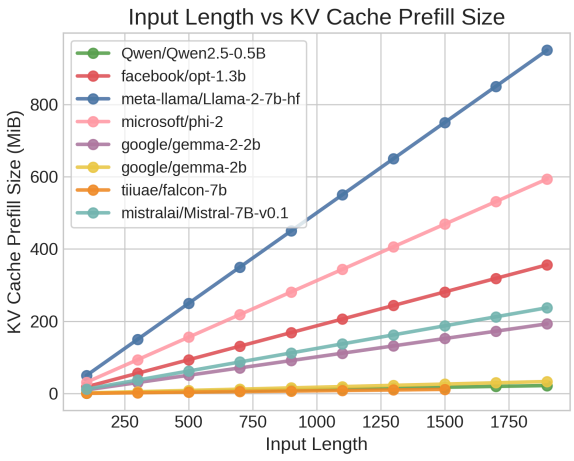


Figure 2: KV cache size vs input length before generation.

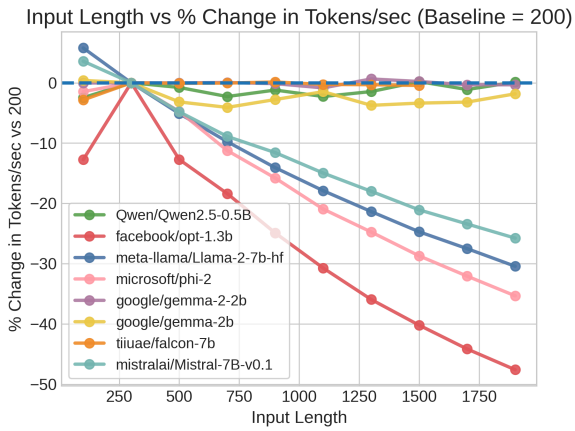


Figure 3: Token generation speed vs input length normalized to 200 tokens.

4.2 Needle-in-a-Haystack Benchmark

Figure 4 shows Needle-in-a-Haystack retrieval accuracy across models under varying KV cache quantization levels.

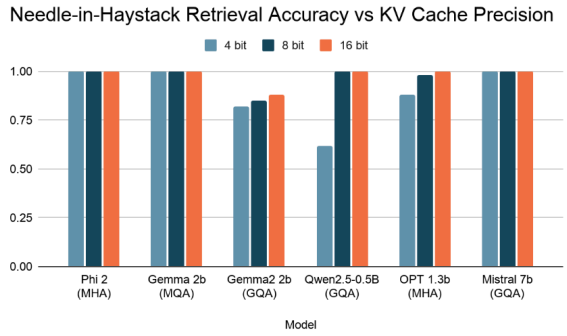


Figure 4: Needle-in-a-Haystack retrieval accuracy across models under different KV cache quantization levels.

5 Discussion

Our performance benchmarks reveal clear trends across different attention mechanism families. First, examining KV cache memory usage, we observe that the cache size grows linearly with input sequence length and the number of generated tokens for all models, which matches theoretical expectations since each token requires storing key and value representations.

However, the scale of this growth differs significantly across attention architectures as shown in Figure 2. Models using MHA, such as Facebook OPT-1.3B, Meta Llama-2-7B, and Microsoft Phi-2, consume the most KV cache memory. Because each attention head maintains its own key and value projections, memory usage scales proportionally with the number of heads, leading to substantially larger cache sizes as sequence length increases.

Models using GQA, including Mistral-7B and Google Gemma-2-2B, demonstrate intermediate memory usage. These architectures share key-value representations across groups of query heads, which reduces the number of stored KV tensors while preserving much of the modeling capacity of MHA. As a result, their memory footprint grows linearly as well, but at a noticeably lower rate than the MHA models.

In contrast, MQA models such as Falcon-7B and Google Gemma-2B show dramatically lower KV cache memory usage. Because all query heads share a single key-value pair, the memory requirements are significantly reduced. In our experiments, this results in KV cache sizes that remain an order of magnitude smaller than those of GQA and MHA models, even as sequence length increases.

Regarding inference throughput, we observe related trends across attention families in Figure 3. For most MHA and GQA models, including OPT-1.3B, Llama-2-7B, Phi-2, and Mistral-7B, the number of tokens processed per second decreases as input sequence length increases. This decline likely arises from the growing KV cache size and increasing memory bandwidth requirements during attention computation.

Interestingly, MQA models maintain nearly constant throughput across sequence lengths, indicating that the reduced KV cache size alleviates memory pressure during inference. Additionally, some GQA models, particularly Gemma-2-2B and Qwen-2.5-0.5B, also show relatively stable throughput. For Qwen in particular, this behavior is likely influenced by its much smaller parameter size, which reduces overall compute and memory demands compared to larger models.

The Needle-in-a-Haystack retrieval benchmark does not reveal strong trends across attention mechanism families. Instead, the results suggest that model scale plays a larger role in robustness to KV cache quantization. In Figure 4, larger models such as Mistral-7B and the family of 2B parameter models maintain higher retrieval accuracy even under aggressive KV cache compression, whereas Qwen-0.5B experiences more substantial degradation. The only exception to this trend is Google's Gemma-2-2B model, which underperforms whether or not it is quantized. We hypothesize that this is not a direct result of the model

architecture or any KV cache details, but possibly due to differences in training methodology, which involved a lot of distillation training.

However, the trend for Qwen-0.5B was substantial, experiencing a sharp decline in accuracy. This behavior is consistent with the idea that smaller models operate closer to their representational limits. When quantization noise is introduced, even when applied only to the KV cache rather than the model weights, the reduced representational capacity of smaller models makes them more sensitive to precision loss.

6 Conclusion

In this work, we investigated memory-efficient attention mechanisms for long-context inference in large language models. Through controlled benchmarks, we compared MHA, MQA, and GQA across several models while also evaluating the effects of KV cache quantization.

Our experiments show that attention architecture has a significant impact on inference-time memory consumption. MHA models incur the largest KV cache footprint due to maintaining separate key–value projections for each head, while MQA dramatically reduces memory usage by sharing a single KV pair across query heads. GQA provides an effective compromise, reducing memory requirements while preserving more representational flexibility than MQA.

Throughput results further highlight the practical advantages of memory- efficient designs. Models with smaller KV caches exhibit more stable generation speeds as context length increases, indicating that memory bandwidth becomes a key bottleneck in long-context inference.

Finally, the Needle-in-a-Haystack experiments show that robustness to KV quantization depends more strongly on model scale than on attention architecture. Larger models maintain higher retrieval accuracy under aggressive KV compression, whereas smaller models degrade more rapidly.

Overall, these results clarify the trade-offs between attention architecture, memory efficiency, and inference performance. They suggest that architectural simplifications such as MQA and GQA are promising directions for enabling long-context language model deployment under strict memory constraints.

7 Github Repository

https://github.com/kabir-vats/KV-Cache-Analysis/tree/main

8 Contributions

All three authors contributed equally to this work. Experimental runs were distributed evenly among the team members, with each responsible for preparing data for different models. The manuscript was also written collaboratively, with sections divided equally among the authors.

References

Armen Aghajanyan, Luke Zettlemoyer, and Sonal Gupta. 2021. Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning. In *Annual Meeting of the Association for Computational Linguistics*.

Joshua Ainslie, James Lee-Thorp, Michiel Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245*. arXiv: 2305.13245 [cs.CL].

Hossein Badri, Anis Ait-Mlouk, Marco Canini, and Pascal Frossard. 2023. HQQ: Half-Quadratic Quantization of Large Language Models. *arXiv preprint arXiv:2309.15217*.

Iz Beltagy, Matthew E Peters, and Arman Cohan. 2020. Longformer: The Long-Document Transformer. *arXiv preprint arXiv:2004.05150*.

Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, and Judy Hoffman. 2023. Token Merging: Your ViT but Faster. In *International Conference on Learning Representations*.

Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. 2019. Generating Long Sequences with Sparse Transformers. *arXiv preprint arXiv:1904.10509*.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. In *Advances in Neural Information Processing Systems*.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient Finetuning of Quantized LLMs. In *Advances in Neural Information Processing Systems*.

Woosuk Kwon, Zhuohan Li, Shunyu Zhuang, and others. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. *Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

Zechun Liu, Shijia Wang, Tianlong Chen, Chao Xu, and Ji Lin. 2024. KIVI: A Tuning-Free Asymmetric 2-bit Quantization for KV Cache. *arXiv preprint arXiv:2402.02750*.

Zichang Liu, Yash Desai, Fanyue Liao, Weitao Wang, Victor Hwang, Zhaozhuo Zheng, others, and Xuan Dong. 2023. Scissorhands: Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Noam Shazeer. 2019. Fast Transformer Decoding: One Write-Head is All You Need. *arXiv preprint arXiv:1911.02150*. arXiv: 1911.02150 [cs.LG].

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. *arXiv preprint arXiv:1706.03762*. arXiv: 1706.03762 [cs.CL].

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient Streaming Language Models with Attention Sinks. In *International Conference on Learning Representations (ICLR)*.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Rui Cai, Zihao Wang, and others. 2023. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.